

ATD2 — Simulador de Estruturas de Dados Lineares em C

Integrante:

Thauan Ricardo Martins

Introdução:

Este trabalho teve como objetivo desenvolver um sistema em linguagem C capaz de simular o funcionamento das principais estruturas de dados lineares estudadas durante a disciplina. O programa foi desenvolvido utilizando menus interativos para permitir que o usuário escolha qual estrutura deseja utilizar e realize operações específicas em cada uma delas.

O sistema foi desenvolvido pensando na organização do código, reutilização de funções e facilidade de entendimento durante a apresentação do trabalho.

Estruturas Implementadas

1. Lista Sequencial:

A lista sequencial foi implementada utilizando vetor com capacidade máxima de 100 elementos. Nessa estrutura, os elementos ficam armazenados em posições contínuas da memória.

Foram implementadas as seguintes operações:

- Inserir elemento em uma posição escolhida;
- Remover elemento de uma posição;
- Buscar elemento;
- Exibir todos os elementos;
- Informar quantidade de elementos.

Essa estrutura foi considerada uma das mais simples do trabalho, pois utiliza apenas vetor e controle de índices.

2. Lista Simplesmente Encadeada:

A lista simplesmente encadeada foi implementada utilizando ponteiros e alocação dinâmica de memória.

Cada elemento da lista possui:

- um valor;

- um ponteiro para o próximo elemento.

Operações implementadas:

- Inserção no início;
- Inserção no fim;
- Remoção por valor;
- Busca de elemento;
- Exibição da lista.

Essa estrutura exigiu maior atenção por causa do uso de ponteiros e do controle correto dos nós da lista.

3. Pilha:

A pilha foi implementada seguindo o conceito LIFO (Last In, First Out), onde o último elemento inserido é o primeiro a ser removido.

Operações implementadas:

- Empilhar;
- Desempilhar;
- Consultar topo;
- Exibir pilha;
- Verificar se está vazia.

Foi utilizado vetor com capacidade máxima de 100 posições.

4. Fila:

A fila foi desenvolvida seguindo a regra FIFO (First In, First Out), onde o primeiro elemento inserido é o primeiro a sair.

Operações implementadas:

- Enfileirar;
- Desenfileirar;
- Consultar início;
- Exibir fila;

- Verificar se está vazia.

A implementação foi feita utilizando vetor.

5. Fila Circular:

A fila circular foi implementada para reaproveitar posições vazias do vetor após remoções.

Foram utilizadas variáveis de controle:

- início;
- fim;
- quantidade de elementos.

Operações implementadas:

- Enfileirar;
- Desenfileirar;
- Consultar início;
- Consultar fim;
- Exibir fila;
- Verificar se está cheia;
- Verificar se está vazia.

Foi necessário utilizar o operador % para controlar a circularidade da estrutura.

6. Fila Dupla:

A fila dupla permite inserções e remoções tanto no início quanto no fim da estrutura.

Operações implementadas:

- Inserir no início;
- Inserir no fim;
- Remover do início;
- Remover do fim;
- Consultar início;

- Consultar fim;
- Exibir fila.

Essa estrutura apresentou um pouco mais de dificuldade devido ao controle dos dois lados da fila.

7. Fila de Prioridade:

A fila de prioridade foi implementada considerando a prioridade dos elementos inseridos.

Cada elemento possui:

- valor;
- prioridade.

Elementos com maior prioridade ficam na frente da fila.

Operações implementadas:

- Inserir elemento com prioridade;
 - Remover elemento de maior prioridade;
 - Consultar próximo elemento;
 - Exibir fila ordenada.
-

Principais Dificuldades:

Durante o desenvolvimento deste trabalho, encontrei diversas dificuldades relacionadas à implementação das estruturas de dados em linguagem C. Apesar de compreender os conceitos teóricos apresentados em aula, tive dificuldades na construção prática do código, principalmente na utilização de ponteiros, alocação dinâmica de memória e organização das funções em diferentes arquivos.

Também foram encontrados problemas na configuração do ambiente de desenvolvimento e do compilador, o que dificultou a realização dos testes e correções necessárias durante a implementação.

Por esse motivo, não foi possível concluir todas as estruturas previstas no enunciado dentro do prazo disponível. Entretanto, foi desenvolvida uma versão funcional contendo parte das estruturas estudadas, permitindo aplicar os conceitos fundamentais de listas, pilhas e filas.

Mesmo não conseguindo finalizar integralmente o projeto, o desenvolvimento do trabalho contribuiu para ampliar meus conhecimentos em programação na linguagem C e no estudo das estruturas de dados lineares.

Funcionamento do Sistema:

Ao iniciar o programa, o usuário escolhe qual estrutura deseja utilizar através do menu principal.

Depois disso, o sistema apresenta um submenu contendo as operações disponíveis para a estrutura escolhida.

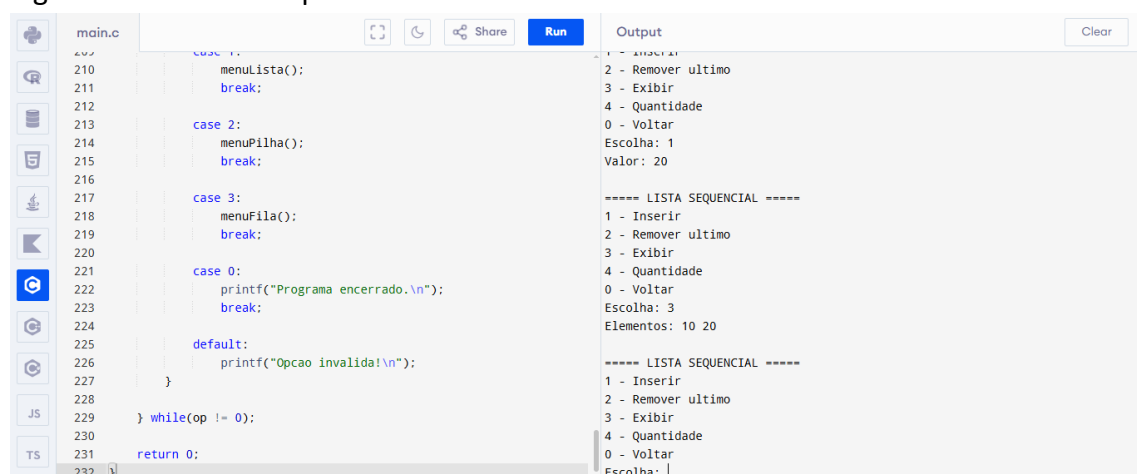
O usuário pode:

- inserir elementos;
- remover elementos;
- consultar informações;
- visualizar os dados armazenados;
- retornar ao menu principal a qualquer momento.

Cada estrutura funciona de forma independente, ou seja, os dados inseridos em uma estrutura não interferem nas outras.

8. Prints da Execução:

Figura 1 – Menu Principal



The screenshot shows a C program running in an IDE. The code on the left is a switch-case menu for a sequential list. The output on the right shows the user selecting option 1 (Inserir), then option 2 (Remover ultimo), then option 3 (Exibir), and finally option 0 (Voltar). The output also shows the list after each operation: after insertion, it has 10 elements; after removal, it has 20 elements. The program then displays the list again and returns to the main menu.

```
main.c
210 menuLista();
211 break;
212
213 case 2:
214 menuPilha();
215 break;
216
217 case 3:
218 menuFila();
219 break;
220
221 case 0:
222 printf("Programa encerrado.\n");
223 break;
224
225 default:
226 printf("Opcao invalida!\n");
227 }
228
229 } while(op != 0);
230
231 return 0;
232 }
```

Output

```
1 - Inserir
2 - Remover ultimo
3 - Exibir
4 - Quantidade
0 - Voltar
Escolha: 1
Valor: 20

----- LISTA SEQUENCIAL -----
1 - Inserir
2 - Remover ultimo
3 - Exibir
4 - Quantidade
0 - Voltar
Escolha: 3
Elementos: 10 20

----- LISTA SEQUENCIAL -----
1 - Inserir
2 - Remover ultimo
3 - Exibir
4 - Quantidade
0 - Voltar
Escolha: |
```

Figura 2 – Lista Sequencial

```
main.c
210 menuLista();
211 break;
212
213 case 2:
214 menuPilha();
215 break;
216
217 case 3:
218 menuFila();
219 break;
220
221 case 0:
222 printf("Programa encerrado.\n");
223 break;
224
225 default:
226 printf("Opcao invalida!\n");
```

Output

```
Escolha: 1
Valor: 100

===== PILHA =====
1 - Empilhar
2 - Desempilhar
3 - Consultar topo
4 - Exibir pilha
0 - Voltar
Escolha: 3
Topo: 100

===== PILHA =====
1 - Empilhar
2 - Desempilhar
3 - Consultar topo
4 - Exibir pilha
0 - Voltar
```

Figura 3 – Pilha

```
main.c
210 menuLista();
211 break;
212
213 case 2:
214 menuPilha();
215 break;
216
217 case 3:
218 menuFila();
219 break;
220
221 case 0:
222 printf("Programa encerrado.\n");
223 break;
224
225 default:
226 printf("Opcao invalida!\n");
```

Output

```
Escolha: 4
Fila: 5 15

===== FILA =====
1 - Enfileirar
2 - Desenfileirar
3 - Consultar inicio
4 - Exibir fila
0 - Voltar
Escolha: 4
Fila: 5 15

===== FILA =====
1 - Enfileirar
2 - Desenfileirar
3 - Consultar inicio
4 - Exibir fila
0 - Voltar
```

Figura 4 – Fila

```
main.c
210 menuLista();
211 break;
212
213 case 2:
214 menuPilha();
215 break;
216
217 case 3:
218 menuFila();
219 break;
220
221 case 0:
222 printf("Programa encerrado.\n");
223 break;
224
225 default:
226 printf("Opcao invalida!\n");
```

Output

```
Escolha: 2
Removido: 5

===== FILA =====
1 - Enfileirar
2 - Desenfileirar
3 - Consultar inicio
4 - Exibir fila
0 - Voltar
Escolha: 4
Fila: 15

===== FILA =====
1 - Enfileirar
2 - Desenfileirar
3 - Consultar inicio
4 - Exibir fila
0 - Voltar
```

Conclusão:

Com a realização deste trabalho foi possível compreender melhor o funcionamento das estruturas de dados lineares estudadas na disciplina.

O desenvolvimento do sistema ajudou no aprendizado sobre:

- vetores;
- ponteiros;
- alocação dinâmica;

- organização de código em C;
- utilização de funções;
- manipulação de dados em memória.

Além disso, o trabalho permitiu desenvolver melhor a lógica de programação e o trabalho individual.

Divisão das Tarefas:

O trabalho foi realizado individualmente.

Responsável: Thauan Ricardo Martins.

Atividades desenvolvidas:

- Planejamento do projeto;
- Desenvolvimento do código;
- Testes e correções;
- Organização dos arquivos;
- Elaboração do relatório.